



Trabajo practico Threads

“Facultad de Ciencia y Tecnología” UADER

Profesores:

- ***Oswaldo Aguiar***
- ***Ulises Rapallini***

Alumnos:

- ***Calivari, Lucas***
- ***Colazo, Ana Lucía***
- ***Laiño, Valentino***
- ***Mas Duarte, Melina***
- ***Mout, Santiago***

Asignatura: *Sistemas Operativos*

Carrera: *Licenciatura en Sistemas de Información*

Año: 2026

1. Tareas simultáneas

Consigna

Crear dos tareas que se ejecuten al mismo tiempo. Implementar un programa donde:

- Un hilo imprima números del 1 al 10.
- Otro hilo imprima letras de la A a la J.
- Ambos deben ejecutarse de forma concurrente.
- Agregar una pequeña pausa entre impresiones para observar la intercalación.

Preguntas para responder:

¿El orden de ejecución es siempre el mismo?

El orden de ejecución no es necesariamente siempre el mismo, aunque puede dar la casualidad que sea el mismo varias veces seguidas, en algún punto dado variará.

¿Quién decide qué hilo se ejecuta primero?

El planificador (scheduler) del sistema operativo es quien decide qué hilo se ejecuta en cada momento, tomando en cuenta varios factores a la hora de elegir:

- Prioridades de los hilos (si están definidas)
- Cuánto tiempo de CPU lleva cada uno Eventos de E/S o llamadas a sleep()
- La cantidad de núcleos disponibles
- Políticas de planificación (round-robin, FIFO, etc.)

¿Qué sucede si se eliminan las pausas?

Si se eliminan las pausas, se ejecuta todo y se obtiene el resultado casi que instantáneamente (en realidad, no es al instante, sino a una velocidad sumamente rápida que hace parecer como si lo fuera.) Suele dar como resultado una secuencia (primero los números y luego letras y viceversa) porque se ejecuta uno antes de darle la oportunidad al otro, pero también se da el caso que algún número o letra quede intercalado.

2. Descarga + interfaz simulada

Consigna

Este ejercicio tiene como objetivo comprender por qué los threads permiten mantener una aplicación activa mientras otra tarea tarda en completarse. Se propone simular una aplicación donde:

- Un hilo representa una descarga de archivo.
- Otro hilo representa una interfaz que sigue respondiendo.
- La descarga debe mostrar progreso: 10%, 20%, 30%, etc.
- La interfaz debe mostrar mensajes como: "Interfaz activa..."

Preguntas para responder:

¿Qué pasaría si la descarga se ejecutara en el hilo principal?

Si la descarga se ejecutará en el hilo principal, la interfaz quedaría congelada sin mostrar mensajes hasta que terminara la descarga completa. La aplicación dejaría de responder durante esos segundos, por eso se usa un hilo separado para tareas lentas de I/O.

¿Por qué es importante esto en aplicaciones gráficas?

En aplicaciones gráficas es fundamental usar hilos (o programación asíncrona) para tareas largas porque el hilo principal debe estar siempre libre para atender eventos de la interfaz de usuario. De lo contrario, la aplicación se congela, no responde a los clics y ofrece una mala experiencia.

¿Este caso es CPU-bound o I/O-bound?

Este caso es I/O-bound, porque la tarea de descarga pasa la mayor parte del tiempo esperando (simulado con `time.sleep`), no realizando cálculos pesados.

En el código, la tarea de "descarga" está simulada con `time.sleep(1)`. En una situación real, descargar un archivo implica esperar datos de la red (o del disco). Durante esa espera, la CPU prácticamente no trabaja; el hilo está bloqueado esperando que lleguen los datos. La tarea de "interfaz activa" solo imprime un mensaje y duerme (`time.sleep(0.5)`), lo que también es espera (I/O simulada hacia la consola, que es un recurso de salida).

3. Productor / Consumidor

¿Por qué se necesita sincronización?

La sincronización es necesaria para que el consumidor pueda utilizar los valores a medida que el productor los va creando, si no estuviesen sincronizados podrían intentar modificar un dato al mismo tiempo, el productor podría seguir creando elementos aunque la cola esté llena o el consumidor trataría de acceder a la cola aunque esté vacía.

¿Qué error aparece si productor y consumidor acceden libremente a la cola?

Si ambos hilos acceden simultáneamente a la cola pueden generar race condition por intentar modificar un mismo elemento al mismo tiempo.

¿Qué ventajas tiene usar una cola thread-safe?

Las colas thread-safe se encargan automáticamente de sincronizar el acceso de los distintos hilos a sus elementos.

4. Race Condition

¿Por qué el resultado puede ser incorrecto?

El resultado puede ser incorrecto porque más de un hilo intenta modificar el contador al mismo tiempo, lo que provoca que solo se refleje un único cambio.

¿Qué es una sección crítica?

Una sección crítica es una parte del código donde los hilos acceden a una variable compartida sin sincronización, o sea una sección donde se puede generar race condition.

¿Qué diferencia hay entre usar lock y operaciones atómicas?

Al utilizar lock, solo un hilo puede ejecutar cierta sección crítica. Mientras que las operaciones atómicas permiten que una variable sea modificada de forma atómica, es decir que nada puede interrumpir esa modificación.

5. Deadlock

Consigna

Aquí el alumno podrá comprender el bloqueo mutuo entre hilos. Crear dos recursos compartidos:

- RecursoA
- RecursoB

Crear dos hilos:

- Hilo 1 bloquea RecursoA y luego intenta bloquear RecursoB.
- Hilo 2 bloquea RecursoB y luego intenta bloquear RecursoA.

Resultado esperado: El programa debe quedar bloqueado para mostrar el deadlock. Luego corregirlo: Aplicar una estrategia para evitar deadlock:

- Bloquear siempre los recursos en el mismo orden.
- Usar timeout.
- Reducir el tiempo dentro de la sección crítica.

Preguntas para responder:

¿Por qué se produce el deadlock?

Porque dos o más hilos quedan esperando indefinidamente recursos que están bloqueados por otros hilos.

¿Qué condiciones deben cumplirse para que exista deadlock?

Deben cumplirse las 4 condiciones de Coffman:

1. *Exclusión mutua*: Un recurso solo puede ser usado por un hilo a la vez.
2. *Retención y espera*: Un hilo mantiene recursos mientras espera otros.
3. *No apropiación*: Los recursos no pueden ser quitados forzosamente.
4. *Espera circular*: Existe un ciclo de hilos esperando recursos entre sí.

Si una de estas condiciones no ocurre, no hay deadlock.

¿Cómo se puede prevenir?

Algunas estrategias:

- Bloquear recursos siempre en el mismo orden.
- Usar timeout al adquirir locks.
- Reducir el tiempo dentro de la sección crítica.
- Evitar mantener múltiples locks simultáneamente.
- Detectar y liberar recursos automáticamente.

6. Starvation

Consigna

Analizar el caso donde un hilo nunca logra acceder a un recurso, para esto crear varios hilos que compitan por un mismo recurso.

- Algunos hilos deben acceder repetidamente al recurso.
- Otro hilo debe intentar acceder, pero quedar postergado muchas veces. 📝 Registrar cuántas veces accede cada hilo.

Preguntas para responder

¿Qué significa que un hilo sufra starvation?

Starvation (inanición) ocurre cuando un hilo queda esperando indefinidamente o durante mucho tiempo porque otros hilos obtienen continuamente el recurso antes que él.

El hilo no está bloqueado permanentemente, pero casi nunca consigue ejecutarse.

¿Es lo mismo que deadlock?

No, ya que en *deadlock* todos los hilos quedan bloqueados lo que hace que ninguno pueda continuar mientras que en *starvation* algunos hilos siguen ejecutándose normalmente y uno o más hilos casi nunca obtienen el recurso.

En starvation el sistema sigue funcionando, pero de manera injusta.

¿Cómo se puede reducir este problema?

Algunas estrategias:

- Planificación justa (*fair scheduling*).
- Uso de colas FIFO para acceso a recursos.
- Limitar el tiempo que un hilo mantiene un lock.
- Dar prioridad a hilos postergados.
- Utilizar semáforos o locks con política de fairness.

El objetivo es evitar que siempre ganen los mismos hilos al competir por recursos.

9. Livelock

Consigna

Un ejercicio importante es comprender una situación donde los hilos no están bloqueados,

pero tampoco avanzan. Simular dos hilos que intentan cederse el paso mutuamente.

Ejemplo:

- Hilo A quiere usar un recurso, pero si ve que Hilo B también quiere usarlo, cede.
- Hilo B hace lo mismo.
- Ambos siguen activos, pero ninguno realiza la tarea.

Preguntas para responder:

¿Por qué livelock no es igual a deadlock?

La diferencia fundamental radica en el estado de ejecución de los hilos y cómo se genera la falta de progreso.

El *Deadlock* (*Interbloqueo*) ocurre cuando dos o más hilos están en un estado de espera bloqueada (bloqueados de forma indefinida) esperando por un recurso que posee otro hilo que está en esa misma situación, es decir, el sistema operativo suspende los hilos (pasan a un estado bloqueado o *waiting*), lo que significa que no consumen ciclos de CPU, pero jamás despertarán por sí mismos. En cambio, en el *Livelock*, los hilos no están bloqueados, sino que se encuentran en un estado activo o ejecutable (*Runnable*) y cambian continuamente de estado interno o ejecutan instrucciones en respuesta mutua, pero sin realizar ningún avance real en la tarea, por lo que el sistema consume activamente ciclos de CPU en un bucle.

¿Los hilos están detenidos?

No, los hilos no están detenidos a nivel de ejecución de hardware. Desde la perspectiva del planificador del sistema operativo, los hilos están activos, en ejecución continua o alternando en estado listo (*Ready*). El problema no es la falta de CPU o de ejecución de instrucciones, sino un bloqueo lógico funcional: el algoritmo está atrapado repitiendo las mismas acciones repetitivas de detección y evasión (como el ejemplo en el que ambos hilos intentan usar un recurso pero lo ceden eternamente al notar la presencia del otro).

¿Cómo se podría resolver?

Existen varias estrategias estándar para mitigar o solucionar un escenario de livelock:

- *Introducción a la aleatoriedad (Backoff aleatorio)*: En lugar de ceder el paso e intentar recuperar el recurso de forma inmediata y sincronizada, se introduce un retardo de tiempo aleatorio antes de que el hilo vuelva a intentar adquirir el recurso (similar al algoritmo CSMA/CD de las redes Ethernet). De esta manera, es altamente probable que uno de los hilos intente tomar el recurso antes que el otro, rompiendo la simetría del bucle.
- *Prioridades dinámicas o asimétricas*: Romper la simetría original asignando prioridades diferentes a los hilos. Si el Hilo A detecta que el Hilo B quiere el recurso, pero A tiene una prioridad mayor, A no cederá (o viceversa), permitiendo que la tarea avance.
- *Mecanismos de detección y cambios de estrategia*: Implementar contadores de reintentos. Si un hilo cede el recurso un número determinado de veces consecutivas sin éxito, cambia su estrategia y fuerza la toma del recurso ignorando educadamente la intención del otro hilo por un ciclo.

10. Sincronización

Consigna

Este ejercicio propone comparar distintos mecanismos de sincronización. Implementando técnicas conocidas de sincronización y control de acceso.

Implementar ejemplos pequeños con:

1. *Mutex / Lock*: Proteger una variable compartida.
2. *Semaphore*: Permitir que sólo 2 hilos accedan a un recurso al mismo tiempo.
3. *Monitor / Condition*: Coordinar productor y consumidor.
4. *Atomic operations*: Incrementar un contador de forma segura.
5. *Critical Section*: Identificar claramente qué parte del código debe ejecutarse de a un hilo por vez.